Week 1, Day 6 (Weekend): CompletableFuture, Streams, Java 8–17 and Self-Mock

Prepared for Upendra Kumar Sharma | 5–6 hours | Main topic: CompletableFuture, Streams edge cases, modern Java features | DSA: Week 1 revision

Today's Schedule

- **0:00–1:00** Study CompletableFuture and write the parallel-calls example (section 1).
- **1:00–2:00** Study Streams, collectors and parallel stream pitfalls (section 2).
- **2:00–2:45** Study Java 8 to 17 features (section 3).
- **2:45–4:00** DSA revision: redo the problems you could not solve cleanly this week (section 4).
- **4:00–5:00** Self-mock: answer 20 Java core questions aloud, timed (section 5).
- **5:00–5:30** Write flashcards for every question you missed, and note your weak topics for Day 7.

1. CompletableFuture

What it is

A Future you can compose. Instead of blocking on get(), you chain steps that run when the previous one completes. It is the standard way to call several services in parallel and combine the results.

Creating

- **supplyAsync(supplier):** runs async and returns a value.
- **runAsync(runnable):** runs async with no result.
- **completedFuture(value):** an already-finished future, useful in tests.
- Without an executor argument these run on ForkJoinPool.commonPool, which is sized for CPU work. **For blocking I/O (database, HTTP), pass your own executor.**

Chaining

- **thenApply(fn):** transform the result, like map.
- **thenCompose(fn):** fn returns another CompletableFuture, and the result is flattened, like flatMap. Use it when the next step is itself async.
- **thenCombine(other, fn):** wait for two independent futures and combine their results.

- **thenAccept / thenRun:** consume the result, or run an action with no result.
- **allOf(futures...):** completes when all are done, but returns CompletableFuture<Void>, so read each result with join().
- **anyOf(futures...):** completes when the first one finishes.
- The Async variants (thenApplyAsync and so on) may run the step on a different thread from the pool.

Error handling

- **exceptionally(fn):** recover from a failure with a fallback value.
- **handle((result, ex) -> ...):** handle both success and failure, and return a value.
- **whenComplete((result, ex) -> ...):** observe the outcome without changing it.
- **Timeouts (Java 9+):** orTimeout(duration) fails the future after the delay, completeOnTimeout(value, duration) completes with a default instead.
- **get() vs join():** get() throws checked exceptions (InterruptedException, ExecutionException). join() throws the unchecked CompletionException. Unhandled exceptions in a chain stay silent until someone calls get or join, so always attach error handling.

```
ExecutorService ioPool = Executors.newFixedThreadPool(10);

CompletableFuture<User> userF = CompletableFuture
    .supplyAsync(() -> userService.find(id), ioPool);
CompletableFuture<List<Order>> ordersF = CompletableFuture
    .supplyAsync(() -> orderService.findByUser(id), ioPool);

CompletableFuture<Profile> profileF = userF
    .thenCombine(ordersF, Profile::new)
    .orTimeout(2, TimeUnit.SECONDS)
    .exceptionally(ex -> Profile.empty());

Profile profile = profileF.join();
```

Collecting many results with allOf

```
List<CompletableFuture<Price>> futures = ids.stream()
    .map(i -> CompletableFuture.supplyAsync(() -> priceService.get(i),
ioPool))
    .toList();

CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
List<Price> prices = futures.stream().map(CompletableFuture::join).toList();
```

2. Streams: Edge Cases and Pitfalls

Basics

- A stream is a pipeline: source, then intermediate operations (filter, map, sorted, distinct), then one terminal operation (collect, forEach, reduce, count).
- Intermediate operations are **lazy**. Nothing runs until the terminal operation is called, and short-circuit operations (findFirst, anyMatch, limit) can stop early.
- A stream can be **consumed only once**. Reusing it throws IllegalStateException.
- sorted() and distinct() are stateful and must see all elements, so they can be costly on large streams.
- peek() is for debugging only. Do not put logic in it.

Collectors

- **toList() vs Stream.toList() (Java 16):** Collectors.toList() gives a mutable ArrayList (not guaranteed). Stream.toList() returns an unmodifiable list.
- **toMap:** throws IllegalStateException on duplicate keys unless you provide a merge function, and throws NullPointerException if a value is null.
- **groupingBy:** group into a Map, with a downstream collector such as counting() or mapping().
- **partitioningBy:** splits into exactly two groups (true and false).
- **joining, summarizingInt, averagingInt** cover common aggregations.

```
// Safe toMap with a merge function
Map<String, Integer> salaryByDept = employees.stream()
    .collect(Collectors.toMap(Employee::dept, Employee::salary,
Integer::sum));

// Count per department
Map<String, Long> headcount = employees.stream()
    .collect(Collectors.groupingBy(Employee::dept, Collectors.counting()));

// Names per department
Map<String, List<String>> names = employees.stream()
    .collect(Collectors.groupingBy(Employee::dept,
             Collectors.mapping(Employee::name, Collectors.toList())));

// Second highest salary
Optional<Integer> second = employees.stream()
    .map(Employee::salary).distinct()
    .sorted(Comparator.reverseOrder()).skip(1).findFirst();
```

Common traps

- **map vs flatMap:** use flatMap when each element produces a stream or collection and you want one flat stream.
- **Primitive streams:** IntStream, LongStream and DoubleStream avoid boxing and provide sum(), average() and max(). average() and max() return Optional types because the stream can be empty.
- **Optional:** use it as a return type, not as a field or parameter. Avoid get() without checking, and prefer orElse, orElseGet (lazy) and orElseThrow.
- **Side effects:** never modify external state inside map or filter. It breaks in parallel streams.
- **Checked exceptions** inside lambdas need a try/catch or a wrapper method.

Parallel streams

- They run on the shared ForkJoinPool.commonPool. One slow task can slow every other parallel stream and async task in the JVM.
- They help only for large data sets with CPU-bound, stateless, independent work. They hurt for small collections, I/O or blocking calls, and ordered or shared-state operations.
- Never accumulate into a shared non-thread-safe list from a parallel stream. Use collect.
- findFirst respects encounter order and findAny does not, so findAny is faster in parallel.
- Measure before choosing parallel. In most backend services, the default is a sequential stream.

3. Java 8 to 17 Features Worth Knowing

Java 8

- Lambdas and functional interfaces (Function, Predicate, Consumer, Supplier, BiFunction), method references.
- Streams API, Optional, default and static methods in interfaces.
- java.time (LocalDate, LocalDateTime, ZonedDateTime, Duration), which replaces the old Date and Calendar.

Java 9 to 11

- Java 9: List.of, Set.of, Map.of factory methods, private interface methods, module system.
- Java 10: **var** for local variable type inference.
- Java 11 (LTS): String methods isBlank, strip, lines and repeat, Files.readString, and the standard HttpClient.

Java 12 to 17

- **Switch expressions (14):** switch returns a value and has no fall-through.
- **Text blocks (15):** multi-line strings with triple quotes, useful for SQL and JSON.
- **Records (16):** concise immutable data classes with generated constructor, accessors, equals, hashCode and toString.
- **Pattern matching for instanceof (16):** no explicit cast after the check.
- **Sealed classes (17, LTS):** restrict which classes can extend or implement a type.
- Helpful NullPointerException messages that name the null variable.

```
// Record
record Point(int x, int y) {}

// Switch expression
String label = switch (day) {
    case SATURDAY, SUNDAY -> "Weekend";
    default -> "Weekday";
};

// Pattern matching for instanceof
if (obj instanceof String s && !s.isBlank()) {
    System.out.println(s.strip());
}

// Sealed hierarchy
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

// Text block
String sql = """
    SELECT id, name
    FROM complaints
    WHERE status = 'OPEN'
    """;
```

Looking ahead to Java 21 (LTS)

- Virtual threads, pattern matching for switch, record patterns and sequenced collections. These come up in interviews for Spring Boot 3 roles, and Week 8 covers them again.

4. DSA Revision (Days 1 to 5)

Redo from scratch, without looking, the problems you needed hints for. Aim to explain the idea out loud before you code.

- **Arrays:** Two Sum, Best Time to Buy and Sell Stock, Contains Duplicate, Product of Array Except Self.
- **Strings:** Valid Anagram, Group Anagrams, Longest Substring Without Repeating Characters, Valid Palindrome.
- **Hashing:** Top K Frequent Elements, Subarray Sum Equals K, Longest Consecutive Sequence.
- **Two pointers:** 3Sum, Container With Most Water, Trapping Rain Water.
- **Sliding window:** Maximum Average Subarray, Minimum Window Substring, Permutation in String.

Method

1. Mark each problem as solved cleanly, solved with a hint, or not solved.
2. Redo every problem not marked "solved cleanly", with a 25-minute timer.
3. For each, write one line on the pattern (for example, "prefix sum plus hashmap") so you can recognize it in new problems.

5. Self-Mock: 20 Java Core Questions (Answer Aloud)

Set a timer for 2 minutes per question and record yourself, or explain to a friend. Score each answer: 2 = complete and confident, 1 = partly right, 0 = could not answer.

1. How does HashMap work internally?
2. Why must equals and hashCode be overridden together?
3. How is ConcurrentHashMap different in Java 7 and Java 8?
4. What are the JVM memory areas?
5. What is the difference between a minor GC and a full GC?
6. When would you choose G1 vs ZGC?
7. Name four causes of memory leaks in Java.
8. ArrayList vs LinkedList, with complexities.
9. What is a fail-fast iterator?
10. Comparable vs Comparator.
11. How do you create an immutable class?
12. Explain the thread states.
13. What do synchronized and volatile each guarantee?
14. Explain happens-before with an example.
15. Why is double-checked locking broken without volatile?
16. How does ThreadPoolExecutor process a task?

17. How do you size a thread pool?
18. What are the four conditions for deadlock, and how do you prevent it?
19. How do you call three services in parallel with CompletableFuture and handle a timeout?
20. When should you use a parallel stream, and when not?

Scoring

- 34–40: strong. Move on, and revisit anything scored 1.
- 26–33: decent. Re-read the day guides for every question scored 0 or 1.
- Below 26: spend extra time on weak topics tomorrow before starting Week 2.

6. End-of-Day Checklist

- [ ] I can chain, combine and time-limit CompletableFutures and explain thenApply vs thenCompose.
- [ ] I can use groupingBy, toMap with a merge function and partitioningBy without looking them up.
- [ ] I can explain when parallel streams help and when they hurt.
- [ ] I can write a record, a switch expression and a sealed interface.
- [ ] I redid the week's weak DSA problems and wrote the pattern name for each.
- [ ] I completed the self-mock and recorded my score and weak topics.

Tomorrow (Day 7, weekend): SOLID principles with code examples, revising your notes and flashcards, and light DSA revision.